

Patrón MVC en el backend

Conclusión

Sí, el backend utiliza el patrón MVC, aunque adaptado al contexto de una API REST con Spring Boot. La estructura de paquetes lo refleja claramente.

Mapeo del patrón MVC

Capa MVC	Paquete/Clase	Responsabilidad
Model	model/Product.java	Entidad JPA que representa un producto (la "M" del patrón).
View	dto/ProductResponseDTO.java	En una API REST, la "vista" es la representación JSON que se devuelve al cliente. Los DTOs de respuesta cumplen este rol.
Controller	controller/ProductController.java	Recibe las peticiones HTTP, delega en el servicio y devuelve respuestas (la "C" del patrón).

Estructura en capas (MVC + capa de servicio)



Detalles importantes

1. Controller (ProductController.java)

Anotado con `@RestController` y `@RequestMapping("/api/products")`. Expone los endpoints CRUD (GET, POST, PUT, DELETE, PATCH) y delega toda la lógica en `ProductService`. No contiene lógica de negocio.

```
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public ResponseEntity<List<ProductResponseDTO>> getAllProducts(
        @RequestParam(required = false) String category,
        @RequestParam(required = false) String name,
        @RequestParam(required = false, defaultValue = "name") String sortBy,
        @RequestParam(required = false, defaultValue = "asc") String sortDir) {
        return ResponseEntity.ok(productService.getAllProducts(category, name, sortBy, sortDir));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponseDTO> getProductById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }
}
```

```

@PostMapping
public ResponseEntity<ProductResponseDTO> createProduct(@Valid @RequestBody
ProductRequestDTO request) {
    ProductResponseDTO created = productService.createProduct(request);
    return ResponseEntity.created(URI.create("/api/products/" +
    created.getId())).body(created);
}

@PutMapping("/{id}")
public ResponseEntity<ProductResponseDTO> updateProduct(@PathVariable Long id,
@Valid @RequestBody ProductRequestDTO request) {
    return ResponseEntity.ok(productService.updateProduct(id, request));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
    productService.deleteProduct(id);
    return ResponseEntity.noContent().build();
}

@PatchMapping("/{id}/active")
public ResponseEntity<ProductResponseDTO> updateActiveStatus(@PathVariable Long id,
@Valid @RequestBody ActiveStatusDTO body) {
    return ResponseEntity.ok(productService.updateActiveStatus(id, body.getActive()));
}
}

```

2. Service (ProductServiceImpl.java)

Anotado con `@Service`. Contiene la lógica de negocio (validaciones, transformaciones, umbral de stock bajo, ordenamiento). Es una capa intermedia entre el controller y el repositorio, lo cual es una **buena práctica** que va más allá del MVC clásico.

```

@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService {

    /** Umbral de stock bajo, según los requisitos funcionales (< 5 unidades). */
    private static final int LOW_STOCK_THRESHOLD = 5;

    private static final Set<String> SORTABLE_FIELDS = Set.of("name", "category", "price",
    "stock", "id");

    private final ProductRepository productRepository;

    @Override
    @Transactional(readOnly = true)
    public List<ProductResponseDTO> getAllProducts(String category, String name, String
    sortBy, String sortDir) {
        String normalizedCategory = StringUtils.hasText(category) ? category.trim() : null;
        String normalizedName = StringUtils.hasText(name) ? name.trim() : null;

        String field = SORTABLE_FIELDS.contains(sortBy) ? sortBy : "name";
        Sort.Direction direction = "desc".equalsIgnoreCase(sortDir) ? Sort.Direction.DESC :
        Sort.Direction.ASC;
        Sort sort = Sort.by(direction, field);

        return productRepository.search(normalizedCategory, normalizedName, sort)
        .stream()
        .map(this::toResponseDTO)
        .collect(Collectors.toList());
    }

    @Override
    @Transactional(readOnly = true)
    public ProductResponseDTO getProductById(Long id) {
        return toResponseDTO(findEntityOrThrow(id));
    }

    @Override
    public ProductResponseDTO createProduct(ProductRequestDTO request) {
        Product product = Product.builder()
        .name(request.getName().trim())
        .description(request.getDescription() != null ? request.getDescription().trim() : null)
    }

```

```

.category(request.getCategory().trim())
.price(request.getPrice())
.stock(request.getStock())
.active(request.getActive() == null || request.getActive())
.build();

return toResponseDTO(productRepository.save(product));
}

@Override
public ProductResponseDTO updateProduct(Long id, ProductRequestDTO request) {
Product product = findEntityOrThrow(id);

product.setName(request.getName().trim());
product.setDescription(request.getDescription() != null ?
request.getDescription().trim() : null);
product.setCategory(request.getCategory().trim());
product.setPrice(request.getPrice());
product.setStock(request.getStock());
if (request.getActive() != null) {
product.setActive(request.getActive());
}

return toResponseDTO(productRepository.save(product));
}

@Override
public void deleteProduct(Long id) {
Product product = findEntityOrThrow(id);
productRepository.delete(product);
}

@Override
public ProductResponseDTO updateActiveStatus(Long id, boolean active) {
Product product = findEntityOrThrow(id);
product.setActive(active);
return toResponseDTO(productRepository.save(product));
}
}

```

3. Repository (ProductRepository.java)

Extiende JpaRepository. Encapsula el acceso a datos (consultas JPA/JPQL).

```

public interface ProductRepository extends JpaRepository<Product, Long> {

@Query("SELECT p FROM Product p " +
"WHERE (:category IS NULL OR LOWER(p.category) = LOWER(:category)) " +
"AND (:name IS NULL OR LOWER(p.name) LIKE LOWER(CONCAT('%', :name, '%')))"
List<Product> search(@Param("category") String category,
@Param("name") String name,
Sort sort);
}

```

4. Model (Product.java)

Entidad JPA con anotaciones @Entity, @Table, etc.

```

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {

@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "products_seq")
@SequenceGenerator(name = "products_seq", sequenceName = "USERS_SEQ", allocationSize =
1)
private Long id;

@Column(nullable = false, length = 100)
private String name;
}

```

```
@Column(length = 500)
private String description;

@Column(nullable = false, length = 100)
private String category;

@Column(nullable = false, precision = 12, scale = 2)
private BigDecimal price;

@Column(nullable = false)
private Integer stock;

@Column(nullable = false)
@Builder.Default
private boolean active = true;
}
```

5. DTOs (dto/)

Separación entre la representación interna (`Product`) y la externa (`ProductRequestDTO` para entrada, `ProductResponseDTO` para salida). Esto evita exponer la entidad directamente.

6. Exception (exception/)

`GlobalExceptionHandler` con `@RestControllerAdvice` para manejo centralizado de errores.

Matiz importante

En una API REST, la "Vista" (View) del MVC clásico no es una plantilla HTML (JSP/Thymeleaf), sino la **serialización JSON** de los DTOs de respuesta. Spring Boot maneja esto automáticamente con Jackson. Además, se añade la capa de **Service** entre Controller y Repository, lo que convierte la arquitectura en un **MVC en capas** (Layered Architecture), que es el estándar recomendado en aplicaciones Spring Boot.

Conclusión final

El backend **sí sigue el patrón MVC**, implementado de forma idiomática para Spring Boot: `Controller` (capa de presentación) → `Service` (lógica de negocio) → `Repository` (acceso a datos) → `Model` (entidad), con DTOs para el intercambio de datos y manejo centralizado de excepciones.